

TetraTHON 2026

Prescreening Track — AgriTech

Product Requirements Document (PRD)

Precision Crop Advisory System & Post-Harvest Loss Reduction Planner

for Smallholder Farmers

Document Version: 1.0

Date: July 17, 2026

Status: Draft — For Pre-Screening Submission

Table of Contents

Table of Contents.....	2
1. Document Control.....	4
2. Executive Summary	4
3. Problem Statement (Recap)	4
3.1 Context	4
3.2 Core Pain Points	5
3.3 Required Prototype Capabilities (from problem statement).....	5
4. Goals & Objectives	5
5. Target Users & Personas	5
5.1 Primary Persona — Smallholder Farmer	5
5.2 Secondary Persona — FPO / KVK Field Agent	6
5.3 Tertiary Persona — Hackathon Evaluator.....	6
6. Scope	6
6.1 In Scope (Prototype / Hackathon)	6
6.2 Out of Scope (Future / Production).....	6
7. Functional Requirements	6
7.1 Module A — Precision Crop Advisory Engine	6
7.2 Module B — Post-Harvest Loss Reduction Planner.....	7
7.3 Cross-Cutting Requirements	7
8. Non-Functional Requirements.....	8
9. System Architecture & Tech Stack	8
9.1 High-Level Architecture	8
9.2 Logical Data Flow.....	9
10. Data Requirements & Sources	9
11. User Flows.....	9
11.1 Advisory Flow	9
11.2 Post-Harvest Flow.....	10
12. Success Metrics Mapped to Evaluation Criteria	10
13. Risks & Assumptions	10
13.1 Risks	10

13.2 Assumptions.....	10
14. Phase-Wise Plan.....	12
Phase 0 — Pre-Screening Prototype (Current Phase)	12
Phase 1 — Hackathon Build (Post-Selection, from July 31, 2026)	12
Phase 2 — Post-Hackathon Roadmap (Pilot → Scale)	12
15. Prototype Plan — What to Build Before the Hackathon	14
15.1 Prototype Philosophy	14
15.2 Minimum Viable Prototype (MVP) Scope	14
15.3 Recommended Tech Stack for Speed	15
15.4 Day-Wise Build Plan (assuming a ~14-day pre-screening window)	15
15.5 What NOT to Build Yet (Avoid Scope Creep)	16
15.6 GitHub Submission Checklist (Highly Recommended by Organisers)	16
15.7 PPT Content Outline (aligned to the 3 rubric criteria).....	17
16. Suggested Team Roles.....	18
17. Appendix.....	18
17.1 Glossary	18
17.2 Reference Data Sources.....	18

1. Document Control

Field	Detail
Project Name	Precision Crop Advisory System & Post-Harvest Loss Reduction Planner
Event	TetraTHON 2026 — Pre-Screening Round
Track	AgriTech (AI/ML, Computer Vision, Open APIs, Data Viz, Market Intelligence)
Document Owner	Team Lead / Product Owner
Version	1.0
Last Updated	July 17, 2026
Next Milestone	Selected teams receive full hackathon problem statement at TetraTHON 2026 Inauguration — July 31, 2026, Navrachana University

2. Executive Summary

India's 85%+ smallholder farmer base (under 2 hectares) is underserved by personalised, timely agronomic advisory, and the country loses an estimated 15–20% of agricultural produce every year to poor storage decisions, delayed market linkage, and lack of real-time price intelligence. This project proposes an integrated AgriTech prototype with two tightly linked modules:

- Crop Advisory Engine — converts simple farmer inputs (location, crop, sowing date, weather, optional leaf photo) into three ranked, confidence-scored, plain-language advisories for the next 7 days covering irrigation, fertiliser application, and pest/disease risk.
- Post-Harvest Decision Planner — converts crop, quantity, storage condition, and location into a Sell / Store / Transport recommendation using mandi price data, spoilage-risk curves, and transport cost modelling, with an optional SMS/WhatsApp price-alert simulation.

The prototype is designed to be demonstrably buildable within the pre-screening window using mocked/open data sources, while the underlying architecture is built to extend cleanly toward a real pilot with an FPO (Farmer Producer Organisation) or Krishi Vigyan Kendra (KVK) post-selection.

3. Problem Statement (Recap)

3.1 Context

Smallholder farmers make high-stakes agronomic and commercial decisions — when to irrigate, what to spray, when to sell, whether to store or transport — largely on intuition, informal advice, or outdated

information. The result is avoidable yield loss on the field and avoidable spoilage/under-pricing after harvest.

3.2 Core Pain Points

- No personalised, localised advisory — generic advice ignores the specific plot, crop stage, and weather.
- No structured decision support at harvest time for Sell vs. Store vs. Transport.
- Limited access to real-time mandi (market) price intelligence.
- Low digital literacy demands a plain-language, low-friction interface rather than a data dashboard.

3.3 Required Prototype Capabilities (from problem statement)

- (a) Ingest location, crop type, sowing date, recent weather observation, and an optional crop/leaf photo → return 3 ranked 7-day advisories (irrigation, fertiliser, pest/disease) with confidence indicators and plain-language rationale.
- (b) Ingest crop type, quantity, storage condition, and location → recommend Sell / Store / Transport using mandi price data, spoilage curves, and transport cost models, with an optional SMS/WhatsApp price-threshold alert simulation.
- Coverage: at least 4 crop types and 5 location combinations.

4. Goals & Objectives

Goal	Objective	Metric
Reduce field-level yield loss	Deliver timely, accurate irrigation/fertiliser/pest advisories	≥3 ranked advisories generated in <3s per request
Reduce post-harvest loss	Give a clear Sell/Store/Transport recommendation with expected return	Recommendation generated for 4+ crops × 5+ locations
Build trust with low-literacy users	Plain-language rationale + confidence indicators	100% of outputs carry a High/Medium/Low confidence tag
Demonstrate scalability	Modular architecture, pluggable data sources	Swap mock→live API with no UI change
Win pre-screening evaluation	Strong problem articulation, approach, and market-fit narrative	PPT + working GitHub prototype submitted on time

5. Target Users & Personas

5.1 Primary Persona — Smallholder Farmer

- Owns/farms under 2 hectares; grows 1–2 crops per season.
- Basic smartphone access, moderate digital literacy, prefers voice/visual over text-heavy dashboards.
- Regional language preference (e.g., Hindi/Gujarati); intermittent connectivity.

5.2 Secondary Persona — FPO / KVK Field Agent

- Assists multiple farmers; needs a slightly more detailed view (batch entry, multiple plots).
- Acts as a trusted intermediary who can relay SMS/WhatsApp alerts.

5.3 Tertiary Persona — Hackathon Evaluator

- Assesses problem understanding, technical approach, and real-world/market viability against the stated evaluation rubric.

6. Scope

6.1 In Scope (Prototype / Hackathon)

- Multi-input intake form for both modules (web-based, mobile-responsive).
- Rule-based / lightweight ML advisory ranking engine with confidence scoring.
- Weather integration (live API where possible, mocked fallback).
- Optional leaf-photo disease classifier using a lightweight pre-trained CNN.
- Mandi price feed (open dataset or synthetic), spoilage-curve model, transport cost model.
- Sell/Store/Transport decision engine with expected-return estimate.
- Data visualisation of spoilage curves and price trends.
- Simulated SMS/WhatsApp price-threshold alert (no real telecom spend required).

6.2 Out of Scope (Future / Production)

- Multilingual voice/IVR interface and full accessibility localisation.
- Real telecom-grade SMS/WhatsApp Business API integration and billing.
- Blockchain-based traceability or payment/escrow integration.
- Full-scale, production-grade ML models trained on proprietary field data.
- Multi-tenant FPO/cooperative management console.

7. Functional Requirements

7.1 Module A — Precision Crop Advisory Engine

ID	Requirement	Priority
FR-A1	Capture location (pin/GPS), crop type, sowing date, recent weather observation, optional leaf photo	Must
FR-A2	Fetch/derive 7-day weather forecast via open API (OpenWeatherMap/IMD) with mocked fallback	Must
FR-A3	Generate 3 ranked advisories: irrigation scheduling, fertiliser application, pest/disease alert	Must
FR-A4	Attach confidence indicator (High/Medium/Low) and plain-language rationale to each advisory	Must
FR-A5	Optional: classify leaf photo for disease/pest signs using a lightweight CNN (transfer learning)	Should
FR-A6	Support at least 4 crop types and 5 location combinations end-to-end	Must
FR-A7	Regional-language toggle for advisory text (stretch goal)	Could

7.2 Module B — Post-Harvest Loss Reduction Planner

ID	Requirement	Priority
FR-B1	Capture crop type, quantity, current storage condition, and farmer location	Must
FR-B2	Ingest mandi price feed (Agmarknet/data.gov.in or synthetic dataset) by crop and nearest market	Must
FR-B3	Compute spoilage-risk / spoilage-curve estimate based on crop type and storage condition	Must
FR-B4	Compute transport cost estimate based on distance to nearest profitable mandi	Must
FR-B5	Recommend Sell / Store / Transport with an expected-return estimate	Must
FR-B6	Visualise spoilage curve and price trend over the decision horizon	Must
FR-B7	Simulate an SMS/WhatsApp alert when price crosses a farmer-set threshold	Should

7.3 Cross-Cutting Requirements

- FR-X1: Session/demo data persistence (SQLite/PostgreSQL) so judges can replay a scenario.
- FR-X2: Seed/demo dataset covering the minimum 4 crops × 5 locations combination matrix.
- FR-X3: Graceful degradation — if a live API is unavailable, fall back to cached/mocked data without breaking the UI.

- FR-X4: Single unified dashboard linking Module A output to Module B input for the same farmer/crop record.

8. Non-Functional Requirements

Category	Requirement
Usability	Plain-language output, iconography over text, minimal fields per screen, mobile-first layout
Performance	Advisory/decision generation under 3 seconds per request on demo hardware
Reliability	Mocked-data fallback for every external API; no hard dependency on live network during demo
Scalability	Stateless service layer; swappable data-source adapters (mock → live) with no UI changes
Security & Privacy	No storage of PII beyond session scope for the prototype; location stored as coarse pin, not exact address
Accessibility	High-contrast UI, large tap targets; regional-language strings externalised for future translation
Maintainability	Modular codebase (intake → engine → decision → visualisation) with clear API boundaries

9. System Architecture & Tech Stack

9.1 High-Level Architecture

Layer	Component	Suggested Technology
Presentation	Farmer intake form + advisory/decision dashboard	React (Vite) or Next.js, Tailwind CSS; mobile-responsive
API / Application	REST API — advisory engine, decision engine, alert simulator	FastAPI (Python) or Node.js/Express
Intelligence	Rule engine + lightweight ML (ranking, spoilage model, leaf classifier)	Python: scikit-learn (rules/ranking), PyTorch/TensorFlow-Lite MobileNetV2 (leaf CNN, stretch)
Data	Weather, mandi prices, crop knowledge base, seed records	OpenWeatherMap/IMD API, Agmarknet/data.gov.in or synthetic CSV, SQLite/PostgreSQL

Layer	Component	Suggested Technology
Messaging (simulated)	Price-threshold alert simulation	Twilio sandbox / mocked webhook + logged message queue
Deployment	Live demo link for evaluators	Vercel/Netlify (frontend), Render/Railway (backend)

9.2 Logical Data Flow

Module A: Intake Form → Weather Adapter (live/mock) → Rule/ML Ranking Engine → Confidence Scoring → Plain-Language Renderer → Dashboard.

Module B: Intake Form → Mandi Price Adapter (live/mock) → Spoilage Model + Transport Cost Model → Decision Engine (Sell/Store/Transport) → Visualisation + Alert Simulator → Dashboard.

Both modules share the same farmer/crop record so a single demo scenario flows end-to-end from advisory to harvest decision.

10. Data Requirements & Sources

Data Type	Primary Source	Fallback for Prototype
Weather (7-day forecast/observation)	OpenWeatherMap API / IMD	Static mocked JSON per location
Mandi prices	Agmarknet (data.gov.in)	Synthetic CSV modelled on historical price patterns
Crop agronomy rules (irrigation/fertiliser/pest windows)	ICAR advisories, KVK crop calendars	Curated static rule table for 4 crops
Leaf disease reference images	PlantVillage dataset (Kaggle)	Small curated sample set, pre-trained model
Transport cost basis	Public fuel-price + distance-based estimate	Simple per-km linear cost function

11. User Flows

11.1 Advisory Flow

1. Farmer opens app, selects/pins location and crop type, enters sowing date.
2. Farmer optionally uploads a leaf photo and/or a recent weather observation.

3. System fetches 7-day forecast and runs the ranking engine.
4. System returns 3 ranked advisories with confidence tags and plain-language rationale.
5. Farmer can tap an advisory to see the reasoning in more detail.

11.2 Post-Harvest Flow

6. Farmer enters crop, quantity, storage condition, and location.
7. System pulls nearby mandi prices and computes the spoilage curve.
8. System computes transport cost to the best nearby market.
9. Decision engine returns Sell / Store / Transport with an expected-return estimate and visual chart.
10. Farmer can set a price threshold; system simulates an SMS/WhatsApp alert when crossed.

12. Success Metrics Mapped to Evaluation Criteria

Evaluation Criterion (from brief)	How This PRD Addresses It
Understanding of the Problem Statement	Sections 3–5: root-cause framing of advisory gap + post-harvest loss, backed by the 15–20% loss and 85% smallholder figures, plus explicit persona definition
Proposed Approach	Sections 7–11: concrete functional requirements, architecture, tech stack, and data flow with a clearly phased, buildable plan
Market Fit & Real-World Relevance	Section 4 (goals), Section 6 (scope discipline), Section 14 (phased scale-up toward FPO/KVK pilot) show a path from prototype to a viable, scalable product

13. Risks & Assumptions

13.1 Risks

- Public API rate limits or downtime during live demo → mitigate with cached/mockered fallback (FR-X3).
- Leaf-disease classifier accuracy will be limited on a small training sample → clearly label as a confidence-scored, non-diagnostic aid.
- Mandi price data granularity may not match every location → use nearest-market approximation with a stated assumption.
- Time pressure before submission may force cutting the leaf-classifier or SMS-simulation scope first (both are marked "Should/Could" priority for this reason).

13.2 Assumptions

- Internet connectivity is available for the live demo; offline-first mode is a future-phase concern.
- SMS/WhatsApp alerting is simulated (logged, not sent via paid telecom channel) for the prototype.
- Judges will evaluate against the stated rubric: Problem Understanding, Approach, Market Fit.

14. Phase-Wise Plan

The plan spans three phases aligned to the TetraTHON 2026 process: the pre-screening prototype (now), the hackathon build (post-selection, from the July 31 inauguration), and a post-hackathon scale-up roadmap.

Phase 0 — Pre-Screening Prototype (Current Phase)

Item	Detail
Duration	From today until the pre-screening submission deadline (typically 10–14 days for such rounds)
Objective	Prove the concept end-to-end with mocked/open data; produce a compelling PPT + visible GitHub progress
Key Deliverables	1) Submission PPT (Problem Understanding, Approach, Market Fit) 2) Public GitHub repo with regular commits 3) Clickable/working demo (local or deployed) 4) Optional short demo video/GIF
Success Criteria	Rubric-aligned PPT; working demo of both modules on at least 4 crops × 5 locations; visible, incremental commit history

Phase 1 — Hackathon Build (Post-Selection, from July 31, 2026)

Item	Detail
Duration	The official hackathon window (length to be confirmed with the full problem statement at inauguration)
Objective	Extend the pre-screening prototype into the full brief received at inauguration; harden the weaker "Should/Could" features
Key Deliverables	Live-API integration (replace remaining mocks), leaf-disease classifier if not already built, polished UI/UX, judge-facing live demo, pitch deck refresh
Success Criteria	All Must-priority FRs live with real data; at least one Should-priority feature (leaf classifier or SMS simulation) fully working; stable live deployment

Phase 2 — Post-Hackathon Roadmap (Pilot → Scale)

Item	Detail
Duration	1–6 months post-event (indicative)
Objective	Validate with a real FPO/KVK pilot; move from rule-based to data-trained models

Item	Detail
Key Deliverables	Pilot with one FPO/KVK cluster, real farmer feedback loop, trained ML advisory model on real field data, real SMS/WhatsApp Business API integration, regional-language UI
Success Criteria	Measurable reduction in reported spoilage/loss in pilot cohort; advisory adoption rate; positive qualitative feedback from field agents

15. Prototype Plan — What to Build Before the Hackathon

This section answers the practical question: given the pre-screening submission is a PPT (with an optional but recommended GitHub prototype), what should the team actually implement now — at prototype, not production, fidelity?

15.1 Prototype Philosophy

- Build breadth over depth: cover both modules end-to-end with mocked data rather than perfecting one module.
- Favour rule-based logic over heavy ML for the advisory ranking and spoilage model — it is fast to build, easy to explain to judges, and still demonstrates sound reasoning.
- Treat the leaf-disease CNN and SMS/WhatsApp simulation as stretch goals — attempt only after the core Must-priority flow works end-to-end.
- Optimise for demo-ability: a clean, working, deployed demo beats a partially working but ambitious one.

15.2 Minimum Viable Prototype (MVP) Scope

Component	Prototype-Level Implementation
Intake Form	Single-page form: location dropdown/pin (5 seeded locations), crop dropdown (4 seeded crops), sowing date picker, weather-observation text/select, optional photo upload
Weather Data	Mocked JSON per seeded location (simulate a 7-day forecast); wire to OpenWeatherMap only if time permits
Advisory Engine	Rule table per crop (irrigation/fertiliser/pest windows by crop stage) + simple scoring function producing 3 ranked outputs with High/Medium/Low confidence and a templated plain-language sentence
Leaf Classifier (stretch)	Pre-trained MobileNetV2 fine-tuned on a small PlantVillage subset for 1–2 crops only, or a clearly-labelled mocked classification if time runs out
Post-Harvest Intake	Form: crop, quantity, storage condition (dropdown: open/warehouse/cold-storage), location
Mandi Price Data	Synthetic CSV with per-crop, per-market daily price series (5 markets) — realistic enough to visualise a trend
Spoilage Model	Simple decay-curve function per crop/storage-condition combination (e.g., exponential or piecewise-linear decay)
Transport Cost Model	Linear per-km cost function using straight-line distance between seeded locations and markets

Component	Prototype-Level Implementation
Decision Engine	Compare expected net return for Sell-now / Store-N-days / Transport-to-market and return the best option with the estimate
Visualisation	Line chart: spoilage curve and price trend over the decision horizon (Chart.js / Recharts)
Alert Simulation (stretch)	On-screen simulated "WhatsApp/SMS" message log when a mock price threshold is crossed — no real telecom integration needed
Dashboard	One connected screen showing both modules for a chosen farmer/crop scenario

15.3 Recommended Tech Stack for Speed

- Frontend: React (Vite) + Tailwind CSS — fast to scaffold, responsive by default.
- Backend: FastAPI (Python) — pairs naturally with the ML/rule-engine code and auto-generates API docs for the judges' benefit.
- Data: SQLite for seed data + CSV files for mandi/weather mocks — no external DB setup overhead.
- Charts: Recharts or Chart.js for spoilage/price visualisation.
- Deployment: Vercel (frontend) + Render/Railway (backend) for a live shareable link.
- Alternative fast path if the team is small: a single Streamlit app can demo both modules with far less boilerplate — trade-off is a less polished UI.

15.4 Day-Wise Build Plan (assuming a ~14-day pre-screening window)

Days	Focus	Output
Day 1–2	Research & finalise rule tables (crop stages, irrigation/fertiliser/pest windows for 4 crops); finalise 5 seed locations; set up GitHub repo + project skeleton	Repo initialised, README, rule tables in CSV/JSON, architecture diagram drafted
Day 3–5	Build Module A: intake form, mocked weather adapter, rule-based ranking engine, confidence scoring, plain-language templates	Module A working end-to-end on mocked data
Day 6–8	Build Module B: intake form, synthetic mandi price data, spoilage model, transport cost model, decision engine	Module B working end-to-end on mocked data
Day 9–10	Integrate both modules into one dashboard; add spoilage/price charts; polish UI/UX and plain-language copy	Connected, demoable dashboard

Days	Focus	Output
Day 11	Stretch goals if time allows: leaf-disease classifier on 1–2 crops, SMS/WhatsApp alert simulation, live weather/mandi API swap-in	At least one Should-priority feature live
Day 12	Deploy live demo (Vercel/Render); write final README with setup instructions and screenshots; record short demo video/GIF	Live shareable link + polished repo
Day 13	Build the submission PPT: Problem Understanding, Approach (architecture diagram, tech stack, workflow), Market Fit & Real-World Relevance	Draft PPT complete
Day 14	Rehearse the narrative, dry-run the demo, final QA pass on both flows across all 4×5 crop/location combinations, submit PPT + GitHub link	Final submission

15.5 What NOT to Build Yet (Avoid Scope Creep)

- Do not integrate paid/production SMS or WhatsApp Business APIs — a simulated log is sufficient and safer.
- Do not attempt full multilingual voice/IVR support — mention it as a roadmap item in the PPT instead.
- Do not train a large custom CNN from scratch — use transfer learning on a small subset, or mock the classifier if time is short.
- Do not build user authentication/accounts — a single-session demo flow is sufficient for pre-screening.
- Do not over-invest in a production-grade database — CSV/SQLite is enough to demonstrate the logic.

15.6 GitHub Submission Checklist (Highly Recommended by Organisers)

- Public repository with a clear README: problem statement summary, architecture diagram, setup/run instructions, and screenshots.
- Regular, visible commits from Day 1 through submission — avoid a single large last-minute commit.
- A short CONTRIBUTING or team-roles note showing who worked on what (demonstrates team collaboration to evaluators).
- Tag or note the demo/live link prominently at the top of the README.
- Optional: a short screen-recording (GIF/MP4) embedded in the README for evaluators who don't run the code.

15.7 PPT Content Outline (aligned to the 3 rubric criteria)

11. Title slide + team introduction.
12. Problem Understanding: the 85% smallholder / 15–20% post-harvest-loss framing, root causes, and why both advisory and post-harvest decisions must be solved together.
13. Proposed Approach: architecture diagram, tech stack, data sources (with mock vs. live called out), and the two user flows.
14. Prototype Demo: screenshots or embedded GIF of both modules working.
15. Market Fit & Real-World Relevance: target users, scalability path (Phase 2 roadmap), and viability (low-cost mocked-to-live data swap, FPO/KVK distribution channel).
16. Roadmap slide: Phase 0 → Phase 1 → Phase 2 summary.
17. Closing slide: GitHub link, live demo link, thank-you/contact.

16. Suggested Team Roles

Role	Responsibility
Product / Research Lead	Owns problem framing, rule-table research (agronomy + market), PPT narrative
Frontend Engineer	Intake forms, dashboard, charts, mobile-responsive UI
Backend / API Engineer	REST API, rule engine, decision engine, data adapters (mock ↔ live)
ML / Data Engineer	Leaf-disease classifier (stretch), spoilage/scoring model tuning, seed data curation
Presentation / Business Lead	Market-fit narrative, deployment, demo rehearsal, GitHub README polish

17. Appendix

17.1 Glossary

- Mandi — a regulated wholesale agricultural produce market in India.
- FPO — Farmer Producer Organisation, a collective of farmers registered as a body for aggregation and market linkage.
- KVK — Krishi Vigyan Kendra, a district-level farm science centre that provides agricultural extension services.
- Spoilage curve — a model of produce quality/value decay over time under given storage conditions.

17.2 Reference Data Sources

- OpenWeatherMap API — <https://openweathermap.org/api>
- India Meteorological Department (IMD) — <https://mausam.imd.gov.in>
- Agmarknet (mandi prices) — <https://agmarknet.gov.in>
- data.gov.in — Government of India Open Government Data Platform
- PlantVillage Dataset (leaf disease images) — available via Kaggle

— End of Document —